The advantage of this approach is that the log processor can now watch the database writer and decide what to do when it receives a `Terminated` or `ChildFailed` signal. It can make an informed decision and, for example, create a new database writer with an alternative URL to a completely different database node.

The `LogProcessingGuardian` no longer has to monitor the entire application, only the `FileWatcher` actors. If the `LogProcessingGuardian` monitors both the file watcher and the database writer, you must distinguish between the signals of a terminated file watcher and a database writer, resulting in less isolated code to handle problems of the different components.

Sketching the design

Let's sketch this design—the better one—in an implementation that compiles. Listing 5.2 shows the `App` and `LogProcessingGuardian`. This design follows creation chain option 2: the guardian creates one file watcher per directory. Each file watcher creates one log processor per file, and each log processor processes one file. The first actor is the `LogProcessingGuardian`, to which `App` passes the directories to be processed.

Listing 5.2 Building the supervisor hierarchy

```
object LogProcessingImprovedDesign extends App {

  val directories = Vector("file:///source1/", "file:///source2/")    ❶

  val guardian = ActorSystem[Nothing](                                ❷
    LogProcessingGuardian(directories),
    "log-processing-app")
```